

category: avalonia

tags: [migration-parity, loading-flow, bootstrap, retry-loop, winforms-port]

severity: high

created: 2026-07-16

updated: 2026-07-16

project-origin: parking-v8-app-avalonia

Migrate từ WinForms sang Avalonia để làm rút nhánh lỗi/retry ngầm định (silent behavior drop)

Tình huống gặp phải

Đang kiểm tra luồng Loading (AppBootstrapper/AppStartupLoader) của bản Avalonia so với bản WinForms gốc (`FrmLoading.cs`, project `1.Main/parking-v8-app`), theo yêu cầu user "kiểm tra luồng loading và so với app windows xem đã đúng chưa".

Triệu chứng / Lỗi

Thứ tự 4 bước loading (`LoadDeviceConfig` → `ConnectToController` → `InitLprEngine` → `LoadAccessKeyCollection`) và logic mở cửa sổ kế tiếp (`SelectLaneMode` vs `MainShell`) đã port đúng 100%. Nhưng nhánh lỗi "**chưa tìm thấy computer hiện tại trong hệ thống**" (`Computer is null` sau khi gọi backend) bị thiếu hoàn toàn — không lỗi build, không crash, chỉ lặng lẽ sai hành vi:

- Bản gốc WinForms: dừng lại, đợi 5s, retry vô hạn (đệ quy `GetDeviceConfig()`), chặn không cho vào app cho tới khi admin đăng ký computer.
- Bản Avalonia (trước khi fix): `IsSuccess = false` nhưng code không kiểm tra riêng nhánh này — bước "LoadDeviceConfig" bị kẹt ở trạng thái `Running` mãi mãi, còn pipeline vẫn đi tiếp với `lanes/controllers/cameras` rỗng, cuối cùng vẫn mở `MainShellWindow` với dữ liệu trống.

Nguyên nhân gốc rễ (Root Cause)

Khi port logic tuần tự có nhánh lỗi/retry của WinForms (thường viết dạng đệ quy hoặc vòng lặp trong 1 method lớn kiểu `FrmXxx.cs`) sang kiến trúc service-tách-lớp của Avalonia (`XxxService.TryLoadAsync` trả về snapshot/DTO), rất dễ chỉ port "happy path" (nhánh `IsSuccess == true`) mà quên port nhánh lỗi cụ thể — vì nhánh lỗi đó không gây lỗi biên dịch, không throw exception, chỉ đơn thuần "đi tiếp với dữ liệu rỗng" nên không bị phát hiện qua build hay test nhanh.

Giải pháp

- Thêm field phân loại lỗi cụ thể vào snapshot/DTO (không chỉ 1 `IsSuccess bool` chung), VD: `ComputerNotRegistered` riêng với các lỗi khác (`DeviceResponse null`, exception...) vì mỗi loại cần xử lý retry khác nhau.

2. Ở tầng gọi (`AppStartupLoader`), bọc call trong `while` (`backend.ComputerNotRegistered`)
`{ progress?.Invoke(...); await Task.Delay(5s); backend = await TryLoadAsync(...); }` — dùng vòng lặp thay vì đệ quy để tránh stack overflow khi port pattern đệ quy của WinForms.
3. Luôn đối chiếu **từng nhánh if/else của method gốc** (không chỉ nhánh chính) khi port, đặc biệt các method dài như `FrmXXX.cs` trong WinForms — liệt kê hết các early-return/nhánh lỗi trước khi bắt đầu code Avalonia.

```
var backend = await backendBootstrapService.TryLoadAsync(configuration, apiServer, cancellationToken);
while (backend.ComputerNotRegistered)
{
    progress?.Invoke(new BootstrapProgressUpdate { StepIndex =
LoadDeviceConfigStepIndex, ..., State = BootstrapStepState.Running });
    await Task.Delay(TimeSpan.FromSeconds(5), cancellationToken);
    backend = await backendBootstrapService.TryLoadAsync(configuration, apiServer, cancellationToken);
}
```

Áp dụng lại (How to reuse)

- Khi migrate 1 luồng có tính "orchestrator" (loading, bootstrap, wizard nhiều bước) từ WinForms → Avalonia: liệt kê TẤT CẢ nhánh if/else/early-return của method gốc thành bảng trước khi port, không chỉ đọc lướt happy path.
- Kiểm tra bằng cách: build xanh KHÔNG đủ để xác nhận parity — phải chủ động dò từng nhánh lỗi/edge-case của bản gốc và grep xem bản mới có xử lý tương ứng không.
- Nếu 1 DTO/snapshot chỉ có `bool IsSuccess` chung cho nhiều loại lỗi khác nhau cần retry-cadence khác nhau → dấu hiệu cần tách field cụ thể hơn.

Chú ý / Cạm bẫy (Gotchas)

- ⚠ Nhánh lỗi bị rớt thường KHÔNG gây lỗi build/crash — chỉ lộ ra khi test tình huống thực tế (VD: máy tính chưa được đăng ký trên hệ thống), nên dễ lọt qua review nếu chỉ chạy happy path.
- ⚠ Port đệ quy (`await GetDeviceConfig()` gọi lại chính nó) của WinForms sang Avalonia PHẢI đổi thành vòng lặp `while`, không giữ nguyên đệ quy — tránh stack overflow khi retry kéo dài (WinForms C# có TCO hạn chế, dễ tràn stack qua nhiều giờ retry).

Tham chiếu

- Project liên quan: `parking-v8-app-avalonia` (Bootstrap/AppStartupLoader.cs, Services/BackendBootstrapService.cs) so với `parking-v8-app` bản 1.Main (Forms/FrmLoading.cs)